

Booting TRENTOS-M Systems in QEMU for ZCU102

Page Content

- [Introduction](#)
- [Booting using Arm Trusted Firmware](#)
 - [Description](#)
 - [Prerequisites](#)
 - [Xilinx QEMU](#)
 - [Resources](#)
 - [Required Resource Files](#)
 - [Building a native seL4 system](#)
 - [Booting a TRENTOS-M system on zynqmp](#)
 - [Using a set-up host machine](#)
 - [Using the TRENTOS-M Development Environment](#)
 - [Using the TRENTOS-M SDK](#)
 - [CI support](#)
- [Booting using the bare metal zynqmp FSBL](#)
 - [Description](#)

Introduction

This page is the result of the investigation done in the scope of [SEOS-2287](#) - Getting issue details... **STATUS** and

[SEOS-2290](#) - Getting issue details... **STATUS** issues. The purpose of the page is to document the boot process of TRENTOS-M systems (in 64-bit version) on the ZCU102 target in the (Xilinx) QEMU.

Booting using Arm Trusted Firmware

Description

In this mode, the boot process relies on the MicroBlaze based Power Management Unit unit and the Arm Trusted Firmware to initialize the system before U-Boot is loaded:

- <https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18841724/PMU+Firmware>
- <https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18842107/Arm+Trusted+Firmware>

The workflow consists of two QEMU instances - the first one running the PMU firmware and the second one running the ATF/U-Boot/seL4 system.

Prerequisites

Note: In case you do not want to replicate the set-up from scratch, it is possible to use a modified version of the TRENTOS-M Docker test container together with the seos_test project to boot native and CAmkES systems for **zynqmp**. For the instructions, skip to the chapter [Using the TRENTOS-M Development Environment](#).

Xilinx QEMU

- Install dependencies

Install QEMU installation dependencies

```
sudo apt-get install libglb2.0-dev libpixmap-1-dev checkinstall
```

- Clone, Build and install Xilinx QEMU (this will result in a xilinx-qemu installation in **/opt/xilinx-qemu**)

Clone, build and install Xilinx QEMU

```
mkdir xilinx-qemu
cd xilinx-qemu
git clone ssh://git@bitbucket.hensoldt-cyber.systems:7999/ext/xilinx-qemu.git
cd xilinx-qemu
git checkout release-1

# Note: During the execution you will be prompted to enter the password to create a .deb package
./build.sh

# Install the created .deb package
sudo dpkg -i ../build_output/xilinx-qemu_1-20211141835_amd64.deb
```

Resources

After Xilinx QEMU has been installed, all other prerequisites are different files needed for the boot process in QEMU. These files are available in the resource submodule of TRENTOS-M SDK (<https://bitbucket.hensoldt-cyber.systems/projects/SSDK/repos/sdk-resources/browse>) in the directory zcu102_sd_card. Alternatively, all the files can be built/acquired from their original sources as follows:

- Clone Xilinx QEMU device trees <https://github.com/Xilinx/qemu-devicetrees> and build the project by executing:

Build QEMU Devicetrees

```
# might require installation of the device-tree-compiler
make
```

- Download the Zynq UltraScale+ MPSoC Board Support Package <https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/embedded-design-tools.html> (ZCU102 BSP).
- Download the Petalinux installer <https://www.xilinx.com/support/download/index.html/content/xilinx/en/downloadNav/embedded-design-tools.html>
- Build PetaLinux by executing the following commands:

Build and install Petalinux

```
mkdir petalinux_install
cd petalinux_install
../petalinux-v2020.2-final-installer.run
cd ..
```

- Create a dummy project to build configured u-boot

Create a dummy Petalinux project

```
source petalinux_install/settings.sh
petalinux-create -t project -s xilinx-zcu102-v2020.2-final.bsp -n petalinux_proj
cd petalinux_proj
```

- Configure u-boot

Configure U-Boot inside the created Petalinux project

```
# After the gui settings window opens change the following 2 parameters:
#       bootcmd value = fatload mmc 0 0x200000000 os_image.elf;bootelf 0x200000000
#       delay in seconds before automatically booting = 0
# <Save>
# <Exit>
petalinux-config -c u-boot
```

- Build the configured U-Boot

Build the configured U-Boot

```
petalinux-build -c u-boot
```

Required Resource Files

File Name	Description	Note	Origin
zynqmp-pmu.dtb	Device tree blob used for the PMUFW boot component based on MicroBlaze		Copied from qemu-devicetrees/LATEST/MULTI_ARCH
zcu102-arm.dtb	Device tree blob used for the zcu102		Copied from qemu-devicetrees/LATEST/MULTI_ARCH
pmu_rom_qemu_sha3.elf	PMU ROM image		Copied from petalinux_proj/pre-built/linux/images
pmufw.elf	PMU firmware	https://xilnx-wiki.atlassian.net/wiki/spaces/A/pages/18841724/PMU+Firmware	Copied from petalinux_proj/pre-built/linux/images
bl31.elf	Arm Trusted Firmware binary	https://xilnx-wiki.atlassian.net/wiki/spaces/A/pages/18842107/Arm+Trusted+Firmware	Copied from petalinux_proj/pre-built/linux/images
u-boot.elf	U-Boot elf image		Found in petalinux_proj/images/linux/ after successful configuration and build

Building a native seL4 system

Building a seL4 native system can be achieved using our **seos_tests** project (https://bitbucket.hensoldt-cyber.systems/projects/HC/repos/seos_tests/browse) with a standard build of one of the 2 available native projects. It is first necessary to set the platform to **zynqmp** in the **build.sh** script and then execute one of the following commands depending on the desired system:

Build the native_sel4test system

```
<PATH_TO_SEOS_TESTS_ROOT>/seos_sandbox/scripts/open_trentos_build_env.sh <PATH_TO_SEOS_TESTS_ROOT>/build.sh  
native_sel4test
```

or

Build the native_hello_world system

```
<PATH_TO_SEOS_TESTS_ROOT>/seos_sandbox/scripts/open_trentos_build_env.sh <PATH_TO_SEOS_TESTS_ROOT>/build.sh  
native_hello_world
```

Booting a TRENTOS-M system on zynqmp

Using a set-up host machine

After the build is successfully completed it is necessary to create the SD card image which contains the system image which is then used in the boot process:

Create the SD Card image containing the os_image.elf

```
dd if=/dev/zero of=sd_card.img bs=128M count=1  
mkfs.vfat -F 32 sd_card.img  
mcopy -i sd_card.img os_image.elf :/
```

After the SD card image was created and placed in the directory together with the rest of the resource files, the system can be run using two QEMU instances as follows:

Start the PMU QEMU instance

```
# First instance
/opt/xilinx-qemu/bin/qemu-system-microblazeel \
-M microblaze-fdt -nographic \
-dtb ./zynqmp-pmu.dtb \
-kernel ./pmu_rom_qemu_sha3.elf \
-device loader,file=./pmufw.elf \
-machine-path /tmp/zynqmp_device/
```

Start the ZCU102 QEMU instance

```
# Second instance
/opt/xilinx-qemu/bin/qemu-system-aarch64 \
-M arm-generic-fdt \
-nographic \
-dtb ./zcu102-arm.dtb \
-device loader,file=./bl31.elf,cpu-num=0 \
-device loader,file=./u-boot.elf \
-global xlnx,zynqmp-boot.cpu-num=0 \
-global xlnx,zynqmp-boot.use-pmufw=true \
-machine-path /tmp/zynqmp_device/ \
-drive file=./qemu_image.sd,if=sd,format=raw,index=1
```

Using the TRENTOS-M Development Environment

In order to run native and CAMkES systems in QEMU for the **zynqmp** target locally using the **seos_tests** project, it is necessary to first pull the tweaked **trentos_test** Docker container (either being locally in the HC Devnet or via using the VPN):

Pull and tag the modified trentos_test Docker container

```
docker pull docker:5000/trentos_test:20210316
docker tag docker:5000/trentos_test:20210316 trentos_test:20210316
```

After you have successfully pulled the required docker container, check out the **SEOS-000_zynqmp_support** branch (https://bitbucket.hensoldt-cyber.systems/projects/HC/repos/seos_tests/browse?at=refs%2Fheads%2FSEOS-000_zynqmp_support) and build the desired system (e.g. demo_hello_world) as follows:

Build the desired system using the seos_tests project

```
source/seos_sandbox/scripts/open_trentos_build_env.sh source/build.sh demo_hello_world
```

The system can be run from the **seos_tests** project as follows:

Prepare the test environment and run test_native_sel4test.py

```
<PATH_TO_SEOS_TESTS_ROOT>/seos_sandbox/scripts/open_trentos_build_env.sh <PATH_TO_SEOS_TESTS_ROOT>/build.sh
test-prepare
<PATH_TO_SEOS_TESTS_ROOT>/seos_sandbox/scripts/open_trentos_test_env.sh <PATH_TO_SEOS_TESTS_ROOT>/build.sh test-
run test_demo_hello_world.py
```

Note: Currently the sel4Test project does not fully work, i.e., not all tests pass

[SEOS-2670](#) - Getting issue details...

STATUS

Using the TRENTOS-M SDK

Finally, it is possible to build and run a TRENTOS-M system using only the SDK package generated by the CI. First, it is necessary to pull the tweaked **trentos_test** Docker container as described in the previous section. Then, download the SDK package generated by the **SEOS-000_zynqmp_support** branch pipeline in CI - https://192.168.82.12/job/trentos-m-sdk/job/SEOS-000_zynqmp_support/. Assuming that after you unpacked the package, the directory structure is as follows:

Directory structure after unpacking the sdk package

```
sdk_root_dir
  sdk
```

the **demo_hello_world** system can be built and run as follows (executed inside the `sdk_root_dir`):

Build the demo_hello_world using the sdk

```
sdk/scripts/open_trentos_build_env.sh \
  sdk/build-system.sh \
  sdk/demos/demo_hello_world/src \
  zynqmp \
  build-zynqmp-Debug-demo_hello_world \
  -DCMAKE_BUILD_TYPE=Debug
```

Run the demo_hello_world on zynqmp using the sdk

```
sdk/scripts/open_trentos_test_env.sh python3 \
  sdk/resources/scripts/zynqmp_simulate.py \
  sdk/resources/zcu102_sd_card/ \
  build-zynqmp-Debug-demo_hello_world/images/os_image.elf
```

A more complex demo such as the **demo_iot_app** requires additional parameters in the simulate script and it can be built and run using the following commands (executed inside the `sdk_root_dir`):

Build the demo_iot_app using the sdk

```
sdk/scripts/open_trentos_build_env.sh \
  sdk/build-system.sh \
  sdk/demos/demo_iot_app/src \
  zynqmp \
  build-zynqmp-Debug-demo_iot_app \
  -DCMAKE_BUILD_TYPE=Debug
```

Run the demo_iot_app on zynqmp using the sdk

```
sdk/scripts/open_trentos_test_env.sh python3 \
  sdk/resources/scripts/zynqmp_simulate.py \
  sdk/resources/zcu102_sd_card/ \
  build-zynqmp-Debug-demo_iot_app/images/os_image.elf \
  --proxy_app sdk/bin/proxy_app \
  --tcp_port 4444 \
  --cpt sdk/bin/cpt \
  --config_file sdk/demos/demo_iot_app/src/configuration/config.xml
```

After each simulation, an output directory is created in the directory where the script is executed which contains the log files as well as the binaries that emulate different memory regions of the emulated device that are used for communication between the 2 QEMU instances. Specifically, the output of the zcu102 QEMU instance is outputted to the stdout while the output of the PMU QEMU instance (and the proxy application if used) are piped to the log files. The resulting folder structure is as follows:

Structure of the created simulation output directory

```
sim_out_20210429-112005
  logs
  |           |
  |           |      qemu_pmu_err.txt
  |           |      qemu_pmu_out.txt
  |           |
  |           |      HW
  |           |
  |           |      qemu-memory-_dcache_rpu0@0x50000
  |           |      qemu-memory-_dcache_rpu1@0x40000
  |           |      ...
```

CI support

Because of the fact that **zynqmp** requires a tweaked QEMU version, running tests in CI for this platform requires modifying the TRENTOS-M test Docker container by adding Xilinx QEMU. Since the container already contains two QEMU versions at this point we decided not to include this in the main test container but create a separate container that provides support for **zynqmp**. Related ticket [SEOS-2671](#) - Getting issue details... STATUS.

The test systems therefore run on CI in a separate branch https://bitbucket.hensoldt-cyber.systems/projects/SEOS/repos/sandbox/browse?at=refs%2Fheads%2FSEOS-000_zynqmp_support.

In order to run the tests locally, it is necessary to pull the container **trentos_test:20210316**, checkout the branch **SEOS-000_zynqmp_support** and run one of the supported systems as usual.

Bootling using the bare metal zynqmp FSBL

"First Stage Bootloader (FSBL) for Zynq UltraScale+ MPSoC configures the FPGA with hardware bitstream (if it exists) and loads the Operating System (OS) Image or Standalone (SA) Image or 2nd Stage Boot Loader image from the non-volatile memory (NAND/SD/eMMC/QSPI) to Memory (DDR/TCM /OCM) and takes A53/R5 out of reset. It supports multiple partitions, and each partition can be a code image or a bitstream. Each of these partitions, if required, will be authenticated and/or decrypted.

FSBL is loaded into OCM and handed off by CSU BootROM after authenticating and/or decrypting (as required) FSBL."

In theory, it should be possible to achieve the same and result bootling the sel4Test project on the zcu102 without running the PMU in a separate QEMU instance and without relying on the ATF. This would mean using the FSBL + U-Boot which would then boot the sel4 system image as in the previous case. Currently, this does not work and the behaviour is as follows:

- After QEMU starts there is no output
- Inspecting the QEMU instruction flow using the "-d in_asm -singlestep -D qemu_log.txt" flag shows that the system ends up in a synchronous interrupt during the execution of the FSBL and waits there in an infinite loop
- ToDo: Use the available source code of the FSBL () to try to further diagnose the issue

Description

TBD